ISPGAYA
instituto superior politécnico

Escola Superior de Ciência e Tecnologia

CTeSP em Redes e Sistemas Informáticos

Linguagens de Programação

2025/2026

"Gestão de Pacientes de um Dentista"

Aluno:

Nuno Filipe Martins Da Cunha (2025105653)

Docente: Prof. Jorge Vieira

23 de Junho de 2026

ÍNDICE DE CONTEÚDOS

Índice de Conteúdos **2**
Introdução **3**
1.1 Enquadramento 3
1.2 Objetivos 3
1.3 Estrutura do Relatório 4
2. Estrutura de Dados **5**
2.1 Classe TipoTratamento 5
2.2 Classe Tratamento 5
2.3 Classe Dente 5
2.4 Classe Paciente 6
2.5 Classe GestaoPacientes 6
3. Algoritmos **7**
3.1 Funcionamento Global 7
3.2 Inicialização da Gestão e Tipos Predefinidos 7
3.3 Criação de um Paciente e dos seus Dentes 7
3.4 Inserção de um Paciente 8
3.5 Procura de um Paciente 8
3.6 Alterar Dados de um Paciente 9
3.7 Eliminar um Paciente 9
3.8 Inserção de um Tipo de Tratamento 9
3.9 Adição de um Tratamento a um Dente 10
3.10 Gestão de um Dente 10
3.11 Consulta dos Tratamentos Efetuados de um Paciente 11
4. Manual do Utilizador **11**
4.1 Inserir Paciente (opção 1) 11
4.2 Listar Pacientes (opção 2) 11
4.3 Alterar Paciente (opção 3) 12
4.4 Eliminar Paciente (opção 4) 12
4.5 Inserir Tipo de Tratamento (opção 5) 12
4.6 Listar Tipos de Tratamento (opção 6) 12
4.7 Gerir Dente (opção 7) 12
4.8 Ver Tratamentos Efetuados de um Paciente (opção 8) 12
4.9 Sair (opção 0) 12
5. Conclusões **13**
5.1 Bibliografia / Webgrafia 13

INTRODUÇÃO

1.1 ENQUADRAMENTO

O relatório descreve o trabalho prático desenvolvido no âmbito da unidade curricular de Linguagens de Programação, cujo tema consiste na criação de um programa de gestão de pacientes de um dentista. Pretende-se uma aplicação que permita registar a informação relativa aos tratamentos efetuados por um dentista aos dentes dos seus pacientes.

Cada paciente é caracterizado pelos seus dados pessoais (nome, morada, telemóvel, idade e NIF) e pela sua dentição, composta por trinta e dois dentes. Cada dente possui uma designação anatómica, um estado de existência (uma vez que pode já ter sido retirado) e um histórico de tratamentos. Por sua vez, cada tratamento está associado a um tipo de tratamento previamente definido.

A aplicação foi desenvolvida na linguagem Java, recorrendo ao ambiente de desenvolvimento IntelliJ IDEA, e segue os princípios da programação orientada a objetos.

1.2 OBJETIVOS

O objetivo principal deste trabalho é desenvolver uma aplicação que permita ao dentista gerir os seus pacientes e os respetivos tratamentos. Para tal, o programa deve permitir as seguintes operações:

- Inserir, alterar, listar e eliminar a informação de um paciente;
- Inserir e listar tipos de tratamento;
- Retirar um dente de um paciente;
- Adicionar um tratamento a um determinado dente;
- Indicar quais os dentes que o paciente já não possui;
- Consultar o estado dental de um paciente, ou seja, os dentes retirados e os tratamentos efetuados.

Pretende-se ainda que a aplicação seja robusta e de fácil utilização, estruturada de modo a facilitar a sua manutenção e futura expansão.

1.3 ESTRUTURA DO RELATÓRIO

O relatório encontra-se organizado em cinco capítulos. No capítulo *Introdução* é apresentado o enquadramento do trabalho e são definidos os seus objetivos. No capítulo *Estrutura de Dados* são descritas as classes e estruturas de dados utilizadas no programa. No capítulo *Algoritmos* é apresentado o funcionamento global da aplicação e os algoritmos das funcionalidades. No capítulo *Manual do Utilizador* são descritas as funcionalidades disponibilizadas ao utilizador. Por fim, no capítulo *Conclusões* são apresentadas as principais dificuldades sentidas, as limitações do programa e as melhorias que poderão vir a ser introduzidas.

2. ESTRUTURA DE DADOS

O código foi repartido por três pacotes (packages), seguindo uma organização em camadas que separa as diferentes responsabilidades da aplicação:

- modelo – contém as classes (Paciente, Dente, Tratamento e TipoTratamento);
- servico – contém a classe GestaoPacientes, responsável pela lógica de negócio e pelas operações sobre os dados;
- visualizacao – contém a classe Menu, responsável pela interação com o utilizador através da consola.

A classe Main, que constitui o ponto de entrada da aplicação, encontra-se na raiz do projeto e limita-se a criar e iniciar o menu.

2.1 CLASSE TIPOTRATAMENTO

Representa uma categoria de tratamento disponível na clínica, como por exemplo "Obturação" ou "Extração". É caracterizada por um identificador único (idTratamento) e por uma descrição. O identificador é atribuído automaticamente e não pode ser alterado.

2.2 CLASSE TRATAMENTO

Representa um tratamento concreto aplicado a um dente numa determinada data. Contém uma referência a um objeto TipoTratamento e a data em que o tratamento foi realizado, armazenada através do tipo LocalDate. Uma vez registado, um tratamento não é alterado, pelo que dispõe apenas de métodos de leitura.

2.3 CLASSE DENTE

Representa um dente de um paciente. É caracterizada pelo número do dente (de 1 a 32), pela sua designação anatómica, por um indicador booleano que assinala se o dente ainda existe e por uma lista de tratamentos. Os tratamentos são armazenados numa ArrayList, uma vez que o número de tratamentos de um dente não é conhecido à partida.

Tambem, quando um dente é retirado, este não é removido da lista de dentes do paciente. Em vez disso, o indicador de existência passa a falso. Desta forma, preserva-se o histórico de tratamentos associados ao dente.

2.4 CLASSE PACIENTE

Representa um paciente da clínica. Contém os dados pessoais (nome, morada, telemóvel, idade e NIF), sendo o NIF utilizado como identificador único do paciente. Inclui ainda uma lista de objetos Dente.

No momento da criação de um paciente, o construtor invoca um método interno que cria automaticamente os trinta e dois dentes, todos inicialmente existentes. As designações dos dentes são obtidas a partir de um vetor (array) constante, partilhado por todos os pacientes, uma vez que são iguais para qualquer dentição.

2.5 CLASSE GESTAOPACIENTES

Constitui a camada de serviço e centraliza toda a lógica da aplicação. Mantém uma lista de pacientes e uma lista de tipos de tratamento, bem como um contador utilizado para gerar identificadores únicos para os tipos de tratamento. Disponibiliza, entre outros, métodos para procurar, inserir, listar e eliminar pacientes, inserir e procurar tipos de tratamento, retirar dentes e adicionar tratamentos.

3. ALGORITMOS

3.1 FUNCIONAMENTO GLOBAL

Após o arranque, a aplicação cria a gestão de pacientes (que carrega automaticamente os tipos de tratamento predefinidos) e apresenta repetidamente o menu, executando o método correspondente à opção escolhida, até o utilizador selecionar a opção de saída.

```
Início {
   gestao <- nova GestaoPacientes()
   Repetir {
      mostrarMenu()
      Leia opção
      Conforme (opção) {
         1: inserirPaciente()
         2: listarPacientes()
         3: alterarPaciente()
         4: eliminarPaciente()
         5: inserirTipoTratamento()
         6: listarTiposTratamento()
         7: gerirDente()
         8: verTratamentosEfetuados()
         0: Escreva "A sair..."
         outro: Escreva "Opção inválida"
      }
   } Enquanto (opção <> 0)
   sc.close()
}
```

3.2 INICIALIZAÇÃO DA GESTÃO E TIPOS PREDEFINIDOS

O construtor da gestão inicializa as listas, coloca o contador de identificadores a 1 e insere os tipos de tratamento predefinidos chamando inserirTipoTratamento().

```
GestaoPacientes() {
   pacientes <- nova lista vazia
   tipoTratamentos <- nova lista vazia
   proximoId <- 1
   inserirTipoTratamento("Destartarização")
   inserirTipoTratamento("Obturação")
   inserirTipoTratamento("Extração")
   inserirTipoTratamento("Tratamento de canal")
   inserirTipoTratamento("Branqueamento")
}
```

3.3 CRIAÇÃO DE UM PACIENTE E DOS SEUS DENTES

O construtor do paciente guarda os dados pessoais, cria a lista de dentes e chama criarDentes(), que gera os trinta e dois dentes a partir do vetor de designações.

```
Paciente(nome, morada, telemovel, idade, nif) {
   guardar os dados nos atributos
   dentes <- nova lista vazia
   criarDentes()
}

criarDentes() {
   Para i de 0 até (nomes_dentes.length - 1) {
```

```
      d <- novo Dente(i + 1, nomes_dentes[i])
      dentes.add(d)
   }
}
```

3.4 INSERÇÃO DE UM PACIENTE

O método do menu pede os dados ao utilizador, validando que os campos de texto obrigatórios não ficam em branco, cria o paciente e chama inserirPaciente() da gestão, que verifica a duplicação de NIF através de procurarPaciente().

```
inserirPaciente() {   // no menu
   Escreva "Nome: "
   Leia nome
   Enquanto (nome = "") {
      Escreva "O nome é obrigatório preencher"
      Leia nome
   }
   Escreva "Idade: "; Leia idade
   Escreva "Telemóvel: "; Leia telemovel
   Enquanto (telemovel = "") { Escreva "O telemóvel é obrigatório"; Leia telemovel
}
   Escreva "NIF: "; Leia nif
   Enquanto (nif = "") { Escreva "O NIF é obrigatório"; Leia nif }
   Escreva "Morada: "; Leia morada
   Enquanto (morada = "") { Escreva "A morada é obrigatório"; Leia morada }
   p <- novo Paciente(nome, morada, telemovel, idade, nif)
   Se (gestao.inserirPaciente(p)) {
      Escreva "Paciente inserido"
   } Senão {
      Escreva "Já existe um paciente com esse NIF"
   }
}

inserirPaciente(p) {   // na gestão
   Se (procurarPaciente(p.getNif()) <> nulo) { Devolver falso }
   pacientes.add(p)
   Devolver verdadeiro
}
```

3.5 PROCURA DE UM PACIENTE

Percorre a lista de pacientes e compara o NIF de cada um com o NIF procurado, devolvendo o paciente encontrado ou nulo.

```
procurarPaciente(nif) {
   Para cada p de pacientes {
      Se (p.getNif().equals(nif)) {
         Devolver p
      }
   }
   Devolver nulo
}
```

3.6 ALTERAR DADOS DE UM PACIENTE

Localiza o paciente com procurarPaciente() e, através de um submenu, altera apenas o campo escolhido usando o respetivo método set.

```
alterarPaciente() {
   Escreva "NIF do paciente a alterar: "; Leia nif
   p <- gestao.procurarPaciente(nif)
   Se (p = nulo) { Escreva "Paciente não encontrado"; Terminar }
   Escreva submenu (Nome, Morada, Telemóvel, NIF, Idade)
   Leia opção
   Conforme (opção) {
      1: Leia valor; p.setNome(valor)
      2: Leia valor; p.setMorada(valor)
      3: Leia valor; p.setTelemovel(valor)
      4: Leia valor; p.setNif(valor)
      5: Leia valor; p.setIdade(valor)
      outro: Escreva "Opção inválida"
   }
   Escreva "Alteração concluída"
}
```

3.7 ELIMINAR UM PACIENTE

Chama eliminarPaciente() da gestão, que procura o paciente com procurarPaciente() e, se existir, o remove da lista.

```
eliminarPaciente() {   // no menu
   Escreva "Qual o NIF do paciente? "; Leia nif
   Se (gestao.eliminarPaciente(nif)) {
      Escreva "Paciente eliminado com sucesso"
   } Senão {
      Escreva "Não existe um paciente com esse NIF"
   }
}

eliminarPaciente(nif) {   // na gestão
   p <- procurarPaciente(nif)
   Se (p = nulo) { Devolver falso }
   pacientes.remove(p)
   Devolver verdadeiro
}
```

3.8 INSERÇÃO DE UM TIPO DE TRATAMENTO

Cria o tipo com identificador automático, incrementa o contador e adiciona-o à lista.

```
inserirTipoTratamento(descricao) {
   tipo <- novo TipoTratamento(proximoId, descricao)
   proximoId <- proximoId + 1
   tipoTratamentos.add(tipo)
   Devolver tipo
}
```

3.9 ADIÇÃO DE UM TRATAMENTO A UM DENTE

Localiza o paciente com procurarPaciente(), o dente com getDente() e o tipo com procurarTipoTratamento(). Só quando todos existem cria o Tratamento e o adiciona ao dente com adicionarTratamento().

```
adicionarTratamento(nif, numeroDente, idTipo) {
   p <- procurarPaciente(nif)
   Se (p = nulo) { Devolver falso }
   d <- p.getDente(numeroDente)
   Se (d = nulo) { Devolver falso }
```

```
   tipo <- procurarTipoTratamento(idTipo)
   Se (tipo = nulo) { Devolver falso }
   t <- novo Tratamento(tipo, LocalDate.now())
   d.adicionarTratamento(t)
   Devolver verdadeiro
}
```

3.10 GESTÃO DE UM DENTE

Localiza o paciente e o dente e apresenta um submenu que permite chamar retirar() sobre o dente ou adicionarTratamento() da gestão. Não permite tratar um dente já retirado (verificado com isExiste()).

```
gerirDente() {
   Escreva "NIF do paciente: "; Leia nif
   p <- gestao.procurarPaciente(nif)
   Se (p = nulo) { Escreva "Paciente não encontrado"; Terminar }
   Escreva "Número do dente (1-32): "; Leia numeroDente
   d <- p.getDente(numeroDente)
   Se (d = nulo) { Escreva "Número de dente inválido"; Terminar }
   Escreva submenu (1-Retirar, 2-Adicionar tratamento)
   Leia opção
   Conforme (opção) {
      1: d.retirar(); Escreva "Dente retirado"
      2: {
         Se (NÃO d.isExiste()) {
            Escreva "Esse dente já foi retirado, não pode ser tratado"
         } Senão {
            listarTiposTratamento()
            Escreva "Id do tipo de tratamento: "; Leia idTipo
            Se (gestao.adicionarTratamento(nif, numeroDente, idTipo)) {
               Escreva "Tratamento adicionado"
            } Senão {
               Escreva "Não foi possível adicionar o tratamento"
            }
         }
      }
      outro: Escreva "Opção inválida"
   }
}
```

3.11 CONSULTA DOS TRATAMENTOS EFETUADOS DE UM PACIENTE

Percorre todos os dentes do paciente (getDentes()) e apresenta apenas os retirados ou com tratamentos, mostrando o estado de cada dente e a lista dos seus tratamentos (getTratamentos()).

```
verTratamentosEfetuados() {
   Escreva "NIF do paciente: "; Leia nif
   p <- gestao.procurarPaciente(nif)
   Se (p = nulo) { Escreva "Paciente não encontrado"; Terminar }
   Escreva "Tratamentos efetuados a" + p.getNome()
   Para cada d de p.getDentes() {
      Se (NÃO d.isExiste()) OU (d.getTratamentos() não está vazia) {
         Se (d.isExiste()) { estado <- "Existe" } Senão { estado <- "Retirado" }
         Escreva d.getNumero(), d.getNomeDente(), estado
         Para cada t de d.getTratamentos() {
            Escreva t.getTipo().getDescricao(), t.getDataTratamento()
```

```
            }
        }
    }
}
```

4. MANUAL DO UTILIZADOR

Ao iniciar a aplicação, é apresentado o menu principal com as opções disponíveis. O utilizador seleciona a operação pretendida introduzindo o número correspondente e premindo a tecla *Enter*. Seguidamente descrevem-se as funcionalidades disponibilizadas.

4.1 INSERIR PACIENTE (OPÇÃO 1)

Permite registar um novo paciente. São solicitados o nome, a idade, o telemóvel, o NIF e a morada. Após a introdução dos dados, são automaticamente criados os trinta e dois dentes do paciente. Caso já exista um paciente com o mesmo NIF, o registo não é efetuado.

4.2 LISTAR PACIENTES (OPÇÃO 2)

Apresenta a lista de todos os pacientes registados, indicando, para cada um, o nome, a idade, o NIF e o telemóvel. Se não existirem pacientes, é exibida uma mensagem informativa.

4.3 ALTERAR PACIENTE (OPÇÃO 3)

Permite alterar os dados de um paciente. O utilizador introduz o NIF do paciente a alterar e, em seguida, seleciona qual o campo que pretende modificar (nome, morada, telemóvel, NIF ou idade). Apenas o campo escolhido é atualizado.

4.4 ELIMINAR PACIENTE (OPÇÃO 4)

Remove um paciente da aplicação a partir do seu NIF. Caso não exista nenhum paciente com o NIF indicado, é apresentada uma mensagem de erro.

4.5 INSERIR TIPO DE TRATAMENTO (OPÇÃO 5)

Permite registar um novo tipo de tratamento, indicando apenas a respetiva descrição. O identificador é atribuído automaticamente pela aplicação.

4.6 Listar Tipos de Tratamento (opção 6)

Apresenta todos os tipos de tratamento disponíveis, com o respetivo identificador e descrição. A aplicação inclui já um conjunto de tipos predefinidos, como destartarização, obturação, extração, tratamento de canal e branqueamento.

4.7 Gerir Dente (opção 7)

Permite retirar um dente ou adicionar-lhe um tratamento. O utilizador indica o NIF do paciente e o número do dente (entre 1 e 32) e, de seguida, escolhe a ação pretendida. Não é possível adicionar tratamentos a dentes que já tenham sido retirados.

4.8 Ver Tratamentos Efetuados de um Paciente (opção 8)

Apresenta os tratamentos efetuados do paciente, listando os dentes que foram retirados e os dentes que possuem tratamentos, juntamente com a descrição e a data de cada tratamento.

4.9 Sair (opção 0)

Termina o programa.

5. CONCLUSÕES

O desenvolvimento deste trabalho permitiu consolidar os conceitos fundamentais da programação orientada a objetos, nomeadamente a definição de classes, a utilização de construtores e métodos, o encapsulamento através de atributos privados e métodos de acesso, e a colaboração entre objetos de diferentes classes.

A maior dificuldade que senti foi organizar o programa em camadas. No início, não era óbvio onde colocar cada método, e por vezes hesitei entre incluir determinada lógica nas classes do modelo ou na camada de serviço. Com o desenvolvimento do trabalho, fui compreendendo melhor o papel de cada camada, o que ajudou a manter o código mais claro e organizado.

Como melhorias futuras, propõe-se a introdução de mecanismos de gravação e leitura de dados a partir de uma base de dados, de modo a garantir a persistência da informação, bem como o reforço da validação das entradas do utilizador. Seria igualmente útil disponibilizar a consulta do histórico completo de tratamentos de cada dente e a possibilidade de remover ou corrigir tratamentos previamente registados.

5.1 BIBLIOGRAFIA / WEBGRAFIA

JetBrains. (2024). ***IntelliJ IDEA documentation***. https://www.jetbrains.com/help/idea/

Oracle. (2024). ***Java documentation***. https://docs.oracle.com/en/java/